Put SkyMart online. One step at a time.

GitHub + Netlify, followed by Neon, email, images and SMS.

A plain-language edition of your deployment document.
Prepared 13 November 2026.

First: publish the demo	Later: launch the business app
Steps 1-5 Upload the code, deploy it and connect your domain.	Steps 6-15 Set up the services, finish the integrations and check production.

What is ready now?

The modern website and demo staff tools work. You can host them on Netlify today without database credentials.

What still needs development?

Shared stock, real login, photo uploads and message delivery are not connected yet. Creating service accounts or adding API keys will not complete these features.

How to use this guide

• Follow the numbered steps in order. Each step ends with a checkpoint.

• Commands are for **Windows PowerShell**. Replace text such as YOUR-USERNAME before running a command.

• Keep passwords and API keys private. Never paste them into GitHub source files.

Project folder:

```
C:\Users\SMT\Desktop\loopBound\SkyMart
```

A quick vocabulary: **repository** = your project and change history; **deploy** = publish a version; **environment variable** = a setting stored outside the code; **staging** = a separate place to test.

DEPLOY THE DEMO / STEP 01

Prepare your computer

You need GitHub and Netlify accounts, Git, and Node.js 22. Neon and the messaging accounts can wait until the later steps.

Open the right folder

```
Set-Location 'C:\Users\SMT\Desktop\loopBound\SkyMart'
node --version
npm --version
git --version
git status
git branch --show-current
git remote -v
```

Git is already initialized. When the guide was prepared, the branch was **codex/skymart-foundation** and no GitHub remote was configured.

Check the project before uploading

Stop the development server if it is running. Then run each command separately. Stop and fix any error before continuing.

```
npm ci
npm test
npm run build
npm run typecheck
```

Command	What it does
npm ci	Installs the exact package versions in the lockfile.
npm test	Checks the category and stock rules.
npm run build	Creates a production version of the app.
npm run typecheck	Checks TypeScript errors.

Check before continuing

All four commands finish successfully. The current demo has eight automated domain tests.

DEPLOY THE DEMO / STEP 02

Push your project to GitHub

In GitHub: create a repository named **SkyMart**. Choose private if you want to restrict source access. Leave the new README, license and .gitignore options unchecked.

In PowerShell: stage and review the files. The existing .gitignore excludes dependencies, build output and real .env files.

```
git add .
git diff --cached --stat
git diff --cached
git status --short
```

Include the source, public assets, docs, tests, migrations and package-lock.json. Do not commit passwords, API keys, .env, node_modules or .next.

If Git asks for your identity, run these with your own details:

```
git config user.name "YOUR NAME"
git config user.email "YOUR GITHUB EMAIL"
```

For this initial repository, commit the files, rename the branch to main and connect the empty GitHub repository:

```
git commit -m "Initial SkyMart application"
git branch -m main
git remote add origin https://github.com/YOUR-USERNAME/SkyMart.git
git remote -v
git push -u origin main
```

Complete GitHub sign-in if prompted. If main or origin already exists, inspect it first instead of repeating the setup. Do not force-push to fix an unexpected error.

Check before continuing

Refresh GitHub. Your files should appear on main, with no secret values or generated folders.

DEPLOY THE DEMO / STEP 03

Connect GitHub to Netlify

- Sign in to Netlify and open your team dashboard.
- Choose **Add new project > Import an existing project > GitHub**.
- Allow access to the SkyMart repository and select it.
- Choose **main** as the production branch.

Netlify setting	Use this value
Framework	Next.js
Production branch	main
Base directory	Repository root - leave blank
Build command	npm run build
Publish directory	.next
Node.js version	22
Secrets for the current demo	None required

These build values already exist in netlify.toml. The repository root must contain package.json. Do not use your Windows folder path as the Netlify base directory.

Click **Publish** or **Deploy** and read the build log. Netlify automatically handles Next.js through its adapter. Do not use npm run dev as the build command or add a catch-all redirect to index.html.

Check before continuing

Netlify shows a successful deployment and gives you an address such as https://YOUR-PROJECT.netlify.app.

After this setup, pushes to main normally trigger new deployments. Your local computer does not need to stay on to serve the website.

DEPLOY THE DEMO / STEP 04

Test the hosted demo

Open the Netlify address. Check the pages below and refresh a product page directly to confirm that routing works.

Page	What to check
/	Home page and sample illustrations
/products	Search, categories and stock filtering
/products/urban-glide	Product details
/about	Company introduction
/contact	Demo enquiry form
/account	Account-setup placeholder
/admin	Demo staff tools

Try one complete workflow

• Create a demo enquiry and find it in the staff workspace in the same browser.

• Record a stock change, reload the page and check the history.

• Check the mobile layout and navigation.

• Test the URL in an unauthenticated browser before sharing it. Check Netlify visitor-access settings if the site is private.

Remember how the demo stores data

Each browser keeps its own sample records. Localhost, the Netlify address and a custom domain have separate browser storage. A new browser will not show another browser's edits.

Check before continuing

The demo works online. You can stop here and use it for review while the real backend is developed.

DEPLOY THE DEMO / STEP 05

Add your domain and HTTPS

Do this after the temporary Netlify address works. You need a domain you own and access to its DNS settings.

• In Netlify, open **Domain management > Production domains > Add a domain**.

• Enter the domain and choose a primary address, such as www.YOUR-DOMAIN.

• Open **Pending DNS verification** and copy the exact DNS records Netlify displays.

• Add those records at your DNS provider. Keep existing email records intact.

• Wait for DNS verification and the HTTPS certificate. Test the primary address and the other address redirect.

Two terms you will see

Term	Meaning
DNS	The settings that send visitors from your domain to your hosting provider.
HTTPS certificate	Enables the secure https:// connection used by browsers.

Use the values shown in your dashboard

Do not copy an old Netlify IP address from a tutorial. DNS instructions depend on your domain and its provider.

Keep the demo marked as a preview. Its current search-engine indexing setting is disabled. Turn indexing on only when the public catalogue is ready.

Check before continuing

Your chosen domain loads over HTTPS. Save this address for login redirects and links in emails.

PREPARE PRODUCTION / STEP 06

Separate testing from live use

Choose where the backend will run before entering real business data. Keeping Netlify application functions and Neon close together reduces database round trips.

Option	When to use it
Singapore + Singapore	A possible production choice for Sri Lankan customers. Netlify function-region selection currently needs Pro or Enterprise.
Ohio + Ohio	A practical development arrangement using Netlify's default function region.

Neon Object Storage currently supports Ohio, N. Virginia, Frankfurt and Singapore. Confirm service availability and your plan before creating the project.

Use distinct environments

Environment	Suggested setup
Development	Local code + Neon development branch + fake data
Staging	A stable staging Netlify address + test backend and test users
Production	main branch + live Netlify project + production backend
Pull request previews	Explicitly assigned test backend, or demo mode

A matching name does not create a connection

A Git branch, a Netlify deployment and a Neon branch are separate things. Configure their connections deliberately. Do not give arbitrary previews production database or message credentials.

Check before continuing

You know the exact app URL and Neon branch for development, staging and production.

PREPARE PRODUCTION / STEP 07

Create the Neon database

• In Neon, create or select the SkyMart project in the chosen region.

• Identify the production branch, then create a **development branch**. Start there.

• In the connection dialog, select the development branch, database and role. Copy its pooled and direct connection strings.

Connection	Purpose
Pooled: DATABASE_URL	The running application. Usually contains -pooler in the hostname.
Direct: DATABASE_URL_UNPOOLED	The controlled database migration process.

Create .env only if it does not exist:

```
if (-not (Test-Path .env)) { Copy-Item .env.example .env }
```

Edit that file locally. Replace the following placeholders with the exact development strings from Neon, including their TLS settings:

```
DATABASE_URL="DEVELOPMENT_POOLED_CONNECTION_STRING"
DATABASE_URL_UNPOOLED="DEVELOPMENT_DIRECT_CONNECTION_STRING"
```

Review drizzle/0000_mature_pyro.sql, verify that the URL points to development, then run:

```
npm run db:migrate
```

The initial migration already exists. Use db:generate only after changing the schema. Drizzle currently reads .env, so a value stored only in .env.local will not automatically reach this command.

Check before continuing

The tables appear in Neon. They start empty: the browser sample data does not automatically seed the database.

DEVELOPMENT REQUIRED / STEP 08

Connect data and secure login

This step needs application changes

The current UI still saves data in the browser. Adding DATABASE_URL or enabling Neon Auth does not switch it to a live backend.

Connect shared records

• Replace browser storage with server queries and validated server updates.

• Make the server hide draft products and reject category cycles.

• Use a database transaction for stock: lock the SKU, check the balance, record the movement and update the balance together. Repeated requests must not deduct twice.

Set up Neon Auth

Enable Auth / Better Auth on the development branch. During implementation, install the documented SDK and connect its server/client adapters, auth routes and login pages:

```
npm install @neon/auth
```

The integration uses NEON_AUTH_BASE_URL and NEON_AUTH_COOKIE_SECRET. Generate a private cookie secret locally; use a different one for production:

```
node -e "console.log(require('node:crypto').randomBytes(32).toString('base64'))"
```

Allow the exact trusted app origins on the correct Auth branch. Add verification and password-reset screens. Configure custom SMTP for production and disable production localhost access.

Staff access: check the signed-in identity and staff role on every protected server operation. Create the first owner through a controlled admin process. Customers must not assign their own role.

If staff authenticator-app MFA is required, settle that first: it remains on Neon Managed Auth's documented roadmap and may require another auth implementation.

Check before continuing

Two browsers see shared records. Login/reset work. Anonymous users and customers cannot call staff APIs or read someone else's private records.

PREPARE PRODUCTION / STEP 09

Set up email with Resend

• Create a Resend account and add a domain or sending subdomain you own, such as mail.YOUR-DOMAIN.

• Add the DNS records Resend supplies and wait for verification.

• Create a sending API key. Choose a verified sender and a real monitored Reply-To mailbox.

Authentication email

In Neon Auth's custom SMTP settings, enter:

Field	Value
Host	smtp.resend.com
Port	465 with implicit TLS, or 587 with STARTTLS
Username	resend
Password	Your Resend API key
Sender email	An address under your verified sending domain
Sender name	SkyMart

Enquiry and staff-notification email

These need a separate application integration. Save the enquiry and a notification-outbox record together. A worker sends queued messages using Resend and records retries and failures. The worker and its scheduler still need to be built.

Two email connections

Neon Auth's SMTP setup sends login-related emails. A Netlify RESEND_API_KEY setting is for the future business-email worker; it does not configure Neon Auth automatically.

Check before continuing

Test a verification/reset email and an enquiry email separately. Check delivery logs and the destination of every link.

PREPARE PRODUCTION / STEP 10

Prepare product-photo uploads

Bundled images in public/images already deploy with Netlify. Staff-uploaded photos will need object storage and upload code.

On a development code branch, link the existing Neon project and development branch:

```
npm install -g neon
neon auth
npm install @neon/config
neon link
neon branches list
```

The planned neon.ts configuration can declare this bucket. Merge it with any existing settings:

```
import { defineConfig } from '@neon/config/v1';

export default defineConfig({
  buckets: {
    'product-images': { access: 'public_read' },
  },
});
```

Use public_read only for public catalogue images. Review the target and changes before provisioning:

```
neon config plan
neon deploy
```

This deploys Neon resources, not your website. Use the storage endpoint and credentials Neon supplies. On Netlify, map them to custom SKYMART_STORAGE_* variables; the runtime reserves AWS credential names.

Code still needed: staff authorization, file type/size checks, unique object keys, a configured storage client and saved database references. Direct browser uploads also need the correct signing/CORS flow. Never expose storage secrets.

To use Netlify Image CDN through Next.js, update the plain img elements to next/image and allow the approved image origin.

Check before continuing

Uploaded images survive redeploys and appear in another browser. Anonymous users cannot upload or overwrite files.

PREPARE PRODUCTION / STEP 11

Set up service SMS

Use Notify.lk if you select it as the SMS provider. Start with requested service updates, such as a response to a customer enquiry.

- Create the account and obtain its user ID and API key.
- Request approval for the SkyMart sender ID.
- Arrange a test balance and use phone numbers you control.
- Plan the server settings NOTIFY_USER_ID, NOTIFY_API_KEY and NOTIFY_SENDER_ID.

How the application should send messages

A server-side worker sends an HTTPS POST to the provider. Keep credentials in the request body, normalize recipients to the required 94... format, inspect the response and record failures. Never put the API key in browser code or full request logs.

Use the test sender correctly

Notify.lk offers NotifyDEMO for testing but says not to send OTP content with that sender. Follow its sender-approval instructions.

Build delivery controls

- Queue messages so a provider outage does not lose the customer request.
- Track attempts and retries without producing duplicate messages.
- Set spending limits and monitor the remaining balance.
- Keep email and SMS marketing choices separate and unchecked by default.

Password resets can use email. SMS authentication is not required just to send service messages.

Check before continuing

A test service message reaches your phone, its attempt is recorded and the API key remains private.

PREPARE PRODUCTION / STEP 12

Set the live environment values

In Netlify, open **Project configuration > Environment variables**. Keep production and test values separate. Include Functions scope for runtime settings when scope selection is available; include Builds only where required. Redeploy after changing values.

Variable	Used for
DATABASE_URL	Future application database access
DATABASE_URL_UNPOOLED	Migration runner only; keep in the controlled migration environment
NEON_AUTH_BASE_URL	The matching branch Auth endpoint
NEON_AUTH_COOKIE_SECRET	Private auth cookie secret
RESEND_API_KEY	Business-email worker
EMAIL_FROM / STAFF_NOTIFICATION_EMAIL	Sender and staff-notification destination
NOTIFY_USER_ID / NOTIFY_API_KEY / NOTIFY_SENDER_ID	SMS worker
APP_URL	Canonical address for generated links

Storage variable mapping on Netlify

Proposed variable	Value from Neon
SKYMART_STORAGE_ACCESS_KEY_ID	S3 access key ID
SKYMART_STORAGE_SECRET_ACCESS_KEY	S3 secret key
SKYMART_STORAGE_ENDPOINT	Branch storage endpoint
SKYMART_STORAGE_REGION	Storage region
SKYMART_STORAGE_BUCKET	product-images

Most names above are planned configuration, not implemented settings. The future code must read them explicitly. Never prefix secrets with NEXT_PUBLIC_ or commit them in netlify.toml. If the worker runs elsewhere, configure its credentials there.

Check before continuing

Each enabled feature has working server code and the correct environment values. No secret is included in browser bundles.

GO LIVE / STEP 13

Promote the completed app

Do this only after the real integrations work in staging. A successful demo deployment is not a production-readiness check.

• Review the exact production Neon project and branch, migrations and restore arrangements.

• Apply the tested, versioned migration to production in a controlled release step. A designated operator can use npm run db:migrate with the production direct URL supplied securely.

• Keep the live application on its pooled connection. Use compatible schema changes so the old app still runs during deployment.

• Enter approved categories, products and SKUs. Add opening stock through an audited import or receipt process. Store database prices in minor units.

• Create and test the first owner account. Do not promote browser QA records or publish sample products as real stock.

• Merge tested code to main, wait for Netlify, then verify actual live behavior.

Do not migrate production on every preview build

Keep database generation, resets and unconditional production migrations out of the normal Netlify build command. Preview builds must not modify the live schema.

Replace the demonstration content

Use real product photos, descriptions, prices, company contact details, warranties and returns information. Remove demo labels only when the underlying features are real. Enable indexing for approved public catalogue pages while protecting account and staff content.

Check before continuing

The deployed commit is known, production records are correct, and the first owner can use the live staff workflow.

GO LIVE / STEP 14

Run the launch checklist

Tick these off before using SkyMart for real customer and inventory work. The current demo cannot pass the backend-related items yet.

- ☐ The production commit and deployment version are recorded.
- ☐ Pages and direct product links work over HTTPS on phone and desktop.
- ☐ Product and stock changes appear in independent browsers.
- ☐ The server hides draft products and rejects invalid category relationships.
- ☐ Anonymous users cannot access staff APIs or actions.
- ☐ Customers can access only their own private account/enquiry data.
- ☐ Sign-up, verification, login/logout and password reset work.
- ☐ Competing stock requests cannot oversell; retries cannot double-deduct.
- ☐ Stock history identifies real staff and cannot be silently rewritten.
- ☐ Enquiries survive redeploys; failed notifications can retry.
- ☐ Uploaded images persist and unauthorized uploads fail.
- ☐ Marketing choices are separate, optional and respected.
- ☐ Spam controls, rate limits and secret-safe error logging are enabled.
- ☐ Contact, pricing, availability, warranty and return details are approved.
- ☐ A restore procedure and deployment rollback process are understood.
- ☐ Usage alerts and SMS spending controls are configured.

Check before continuing

Test the whole journey with a customer account and a separate staff account. Record any remaining issue before launch.

MAINTAIN THE SITE / STEP 15

Update and recover safely

For later changes, start a feature branch from an up-to-date main:

```
git switch main
git pull --ff-only
git switch -c codex/your-feature-name
```

After editing, run the checks and review what you are about to commit:

```
npm test
npm run build
npm run typecheck
git add .
git diff --cached
git commit -m "Describe the change"
git push -u origin codex/your-feature-name
```

Create a GitHub pull request. Test its Netlify preview against a test backend, then merge to main. Add CI checks and branch protection as the project grows. The current Netlify build command does not separately run npm test.

If a release breaks the site

In Netlify Deploys, republish a retained known-good deployment. Also revert or fix the Git code so the next automatic deployment does not bring the issue back.

Application rollback is not database recovery

Publishing an older site version does not undo migrations or restore data. Use the separately tested Neon restore process for data incidents, preserving new legitimate records.

Keep an eye on usage

Review Netlify build/function logs, Neon capacity, Resend delivery results and Notify.lk balance/failures. Hosting, database, storage, email, SMS, domain and mailbox charges are separate. Check your actual plans before launch.

Check before continuing

You can publish a tested change, identify the live version and explain how you would recover from a bad release.

REFERENCE / QUICK FIXES

When something goes wrong

Problem	First thing to check
origin already exists	Run git remote -v. Keep the right remote or deliberately change it.
main does not match any	Create the first commit and confirm the branch name.
Netlify cannot find package.json	Base directory and repository-root contents.
Netlify build fails	First real error in the log, Node 22, lockfile and filename casing.
Product-page refresh gives 404	Next.js adapter, .next output and inappropriate SPA/static-export settings.
Neon has no application records	The browser demo is not connected merely by adding a database URL.
Migration URL is missing	DATABASE_URL_UNPOOLED in .env or process environment.
Runtime secret is missing	Correct deployment context, Functions scope and a fresh deploy.
Invalid login domain	Exact trusted origin on the correct Neon Auth branch.
Emails fail	Verified sender/domain, provider logs and the separate Neon SMTP setup.
Images return 403	Access mode, branch credential, endpoint and public/private delivery design.
SMS fails	Sender approval, phone format, balance and provider response.
Reviewers cannot open the site	Netlify visitor-access/private-project settings.

Where to start today

Complete steps 1-4 for a reviewable demo URL. Then finish shared data and authentication before using SkyMart for real stock.

REFERENCE / OFFICIAL DOCUMENTATION

Useful links

These clickable links support the original guide. Provider features and dashboard labels can change; check them again when you implement a service. Documentation was checked for the source guide on 13 November 2026.

01 GitHub: upload existing code

02 Netlify: deploy a Git repository

03 Netlify: Next.js support

04 Netlify: domain DNS setup

05 Netlify: function regions

06 Netlify: environment variables

07 Netlify: function environment restrictions

08 Netlify: deployment management and rollback

09 Neon: pooled and direct database connections

10 Neon: Next.js authentication setup

11 Neon: trusted authentication domains

12 Neon: authentication production checklist

13 Neon: authentication roadmap and MFA status

14 Neon: object storage setup

15 Resend: verify a sending domain

16 Resend: SMTP settings

17 Notify.lk: SMS API and test-sender rules

18 Next.js: environment variables

Source document in the project: docs/deployment-guide.md. This PDF is a simplified reading edition; the Markdown document retains the fuller technical explanations.